

ECE1778: Creative Applications for Mobile Devices

Chen Ying

Assistant Professor, Teaching Stream

Department of Electrical and Computer Engineering

University of Toronto

Last Week's Lecture

Expo Router: File-Based Routing (`app/` directory)

- `index.tsx`, `_layout.tsx`
- `[id].tsx`: Dynamic routes
- `(tabs)/`: Route group

State Management

- **Context API**: Global state without prop drilling
- **useReducer**: Managing complex local state
- **Redux Toolkit**: Scalable global state for larger apps

Example: Theme Switcher

Features

- Show current theme (light/dark)
- Toggle theme using Redux Toolkit

Steps

1. Create a slice (`themeSlice`, `themeReducer`)
2. Configure store (`store`)
3. Wrap app with Provider (`<Provider store={store}>`)
4. Use state & dispatch (`useSelector`, `useDispatch`)

Create a Slice

Define state + reducer logic + actions in one place

redux-demo/features/theme/themeSlice.ts

```
1 import { createSlice } from "@reduxjs/toolkit";
2
3 type ThemeState = { theme: "light" | "dark" };
4
5 const initialState: ThemeState = { theme: "light" };
6
7 const themeSlice = createSlice({
8   name: "theme",
9   initialState,
10  reducers: {
11    toggleTheme: (state) => {
12      state.theme === "light" ? "dark" : "light";
13    },
14  },
15 });
16
17 // Action creators are generated for each case reducer function
18 export const { toggleTheme } = themeSlice.actions;
19
```

Configure Store

Add slice reducers to a centralized Redux store

redux-demo/store/store.ts

```
1 import { configureStore } from "@reduxjs/toolkit";
2 // Default export can be imported with any name
3 import themeReducer from "../features/theme/themeSlice";
4
5 export const store = configureStore({
6   reducer: {
7     theme: themeReducer, // key = slice name, value = reducer
8   },
9 });
10
11 // Infer the `RootState` and `AppDispatch` types from the store
12 export type RootState = ReturnType<typeof store.getState>;
13 export type AppDispatch = typeof store.dispatch;
```

Wrap App with Provider

Make store accessible to all screens

redux-demo/app/_layout.tsx

```
1 import { Stack } from "expo-router";
2 import { Provider } from "react-redux";
3 import { store } from "../store/store";
4
5 export default function RootLayout() {
6   return (
7     <Provider store={store}>
8       <Stack />
9     </Provider>
10  );
11 }
```

Use State & Dispatch

useSelector: Read state from the Redux store

useDispatch: Trigger actions to update state

redux-demo/app/index.tsx

```
1 import { View, Text, Button, StyleSheet } from "react-native";
2 import { useSelector, useDispatch } from "react-redux";
3 import { RootState, AppDispatch } from "../store/store";
4 import { toggleTheme } from "../features/theme/themeSlice";
5
6 export default function HomeScreen() {
7   const theme = useSelector((state: RootState) => state.theme.theme);
8   const dispatch: AppDispatch = useDispatch();
9
10  return (
11    <View
12      style={[styles.container, theme === "light" ? styles.light : styles
13    >
14      <Text style={styles.text}>Current theme: {theme}</Text>
15      <Button title="Toggle Theme" onPress={() => dispatch(toggleTheme())
16    </View>
17  );
18 }
19
```

Live Demo

Nothing changes

- `toggleTheme` reducer doesn't work

Broken Reducer

redux-demo/features/theme/themeSlice.ts

```
1 import { createSlice } from "@reduxjs/toolkit";
2
3 type ThemeState = { theme: "light" | "dark" };
4
5 const initialState: ThemeState = { theme: "light" };
6
7 const themeSlice = createSlice({
8   name: "theme",
9   initialState,
10  reducers: {
11    toggleTheme: (state) => {
12      state.theme === "light" ? "dark" : "light";
13    },
14  },
15 });
16
17 // Action creators are generated for each case reducer function
18 export const { toggleTheme } = themeSlice.actions;
19
```

Looks like it flips the theme, but it does nothing

Broken Reducer

```
const themeSlice = createSlice({
  reducers: {
    toggleTheme: (state) => {
      state.theme === "light" ? "dark" : "light";
    },
  },
});
```

- Calculate "dark" or "light", but does nothing with the result
- State is never updated → UI doesn't change

Broken Reducer

```
const themeSlice = createSlice({
  reducers: {
    toggleTheme: (state) => {
      state.theme === "light" ? "dark" : "light";
    },
  },
});
```

A reducer must either

- Update the state (e.g., assign a new value), or
- Return a new state object

This broken reducer doesn't assign or return anything →
State stays the same

Idiomatic Fix

Mutate the property directly

- Redux Toolkit will handle the update safely

```
toggleTheme: (state) => {  
  state.theme = state.theme === "light" ? "dark" : "light";  
},
```

- Redux Toolkit automatically tracks these updates
- You write simple code, and it stays safe under the hood
- Clear and concise for object-shaped state (like `state.theme`)

Immer

Redux Toolkit uses Immer under the hood

- Immer lets you write code that looks like mutation
- But actually performs safe, immutable updates

```
// Classic Redux (no Immer)
return { ...state, theme: "dark" };

// With Immer (used by Redux Toolkit)
state.theme = "dark";
```

- Easier to read and write
- Still keeps Redux's immutability guarantee

Why Immutability

Redux state is **immutable** — we never modify it directly

- Each update creates a **new state object**
- Make updates **predictable** and easier to debug
- Enable **efficient re-rendering** — React can detect changes by comparing object references
- Power **Redux DevTools** features like time-travel debugging

Immer: Benefits

Redux state is **immutable** — we never modify it directly

Immer helps maintain immutability automatically

- You write simple “mutating” code, but Redux still stays safe and predictable

Redux Toolkit Doc: Writing Reducers with Immer

Fixed Reducer

Redux Toolkit's `createSlice` automatically use Immer

redux-demo/features/theme/themeSlice.ts

```
1 import { createSlice } from "@reduxjs/toolkit";
2
3 type ThemeState = { theme: "light" | "dark" };
4
5 const initialState: ThemeState = { theme: "light" };
6
7 const themeSlice = createSlice({
8   name: "theme",
9   initialState,
10  reducers: {
11    toggleTheme: (state) => {
12      state.theme = state.theme === "light" ? "dark" : "light";
13    },
14  },
15 });
16
17 export const { toggleTheme } = themeSlice.actions;
18 export default themeSlice.reducer;
```

Write simpler immutable update logic using “mutating” syntax

Alternative Fix

Return a new state object instead of modifying it

```
toggleTheme: (state) => {  
  return { theme: state.theme === "light" ? "dark" : "light" };  
},
```

Key idea: A reducer must either

- Update the state (by changing its properties), or
- Return a new one

Theme Switcher Flow

1. Create a slice with `createSlice`
2. Configure store with `configureStore`
3. Wrap app with `<Provider>`
4. Access state and trigger updates in components with `useSelector` + `useDispatch`

Takeaways of Example

- **Redux Toolkit** centralizes global state and logic
- **Slices** organize **state + reducers + actions** cleanly
- **Provider + hooks** allow **any component** to access state
- Mirrors the Context + `useReducer` pattern, but **scales better** for larger apps

Redux Toolkit: Benefits

Compared to Context + `useReducer`

- Handle large-scale global state efficiently
- No prop drilling
- Industry standard — prepares you for real-world apps

Recap: Redux Toolkit

- **Centralized state:** One store holds all app state
- **Slices:** Organize state + reducers + actions
- **Provider + hooks:** Any component can read or update state using `useSelector`, `useDispatch`
- **Predictable updates:** Actions → reducers → new state
- **Scales for large apps:** Handles complex global state better than Context API

Redux Toolkit gives **structure, predictability, and industry-standard patterns** for managing global state

Recap: State Management

- **Context API**
 - Simple global state for medium apps
 - Lightweight, built-in
 - Not ideal for very complex logic or large apps
- **useReducer**
 - Centralize complex state updates
 - Good for multiple related values
 - Can combine with Context for global state

Recap: State Management

- **Redux Toolkit**

- Industry-standard for scalable global state
- Organize state, actions, and reducers with slices
- Predictable updates, no prop drilling, dev tools

Start simple with Context + useReducer

Move to Redux Toolkit as apps grow in complexity

Assignment 4

Advanced State Management and Persistence

- Replace Context with Redux Toolkit
- Add persistent local storage

Today's Lecture

Data Persistence

Async Storage

A simple, persistent, **key-value** storage solution for React Native apps

- **Cross-Platform:** iOS, Android, Web, macOS, and Windows
- **Persistent:** Data remains available across app restarts
- **Simple API:** Save, read, and delete string data with a few methods
- For example: `setItem()`, `getItem()`

Doc: react-native-async-storage.github.io/async-storage

Async Storage: Set Up

Installation

- With npm

```
npm install @react-native-async-storage/async-storage
```

- With Expo CLI

```
npx expo install @react-native-async-storage/async-storage
```

Async Storage: Usage

Async Storage works like a simple **key–value database**

- Each value is stored under a **unique string key**
- Use **descriptive** keys to identify and organize stored data (e.g., `@FitnessTracker_activities`)
- Reusing the same key **overwrites** the previous value
- Async Storage can only store **string** data
- For complex data (arrays or objects), use JSON
 - `JSON.stringify()` when **saving**
 - `JSON.parse()` when **loading**

Async Storage: API

Async Storage provides a simple asynchronous API for key–value operations

- `setItem(key, value)`: Store a string value
- `getItem(key)`: Retrieve a string value (or null if not found)

All operations are asynchronous

- Always use `await` or `.then()` to handle results

Store Data

`setItem(key, value)`: Set a string `value` for given `key`

- Add a new entry if the key doesn't exist
- Overwrite the existing value if the key already exists

```
await AsyncStorage.setItem("@FitnessTracker_activities", "Swimming, Running")
```

If saving objects or arrays, convert to JSON:

```
await AsyncStorage.setItem("@FitnessTracker_activities", JSON.stringify(activ
```

Read Data

`getItem(key)`: Get the string value stored under a given key

- Return a string value for existing key
- Return `null` if the key is not found

```
const value = await AsyncStorage.getItem("@FitnessTracker_activities");
```

For JSON data:

```
const activities = value ? JSON.parse(value) : [];
```

Example: Persist Theme

Extend our theme switcher to save/load from Async Storage

- On app start: Load saved theme
- On toggle: Save new theme

Add Async Storage

```
async-storage-demo/  
├── app/  
│   ├── _layout.tsx  
│   └── index.tsx  
├── features/theme/  
│   └── themeSlice.ts  
├── storage/  
│   └── themeStorage.ts  
├── store/  
│   └── store.ts  
└── ...
```

Separate persistence logic from state management

- Redux slices (`features/`, `store/`) handle in-memory state
- `storage/` manages persistence

Live Coding

Write `storage/themeStorage.ts`

| `async-storage-demo/storage/themeStorage.ts`

```
1 import AsyncStorage from "@react-native-async-storage/async-storage";
2
3 const KEY = "@MyApp_theme";
4
5 export async function loadTheme(): Promise<"light" | "dark"> {}
6
7 export async function saveTheme(theme: "light" | "dark"): Promise<void> {
```

Add Async Storage

async-storage-demo/storage/themeStorage.ts

```
1 import AsyncStorage from "@react-native-async-storage/async-storage";
2
3 const KEY = "@MyApp_theme";
4
5 export async function loadTheme(): Promise<"light" | "dark"> {
6   const saved = await AsyncStorage.getItem(KEY);
7   return saved ? (saved as "light" | "dark") : "light";
8 }
9
10 export async function saveTheme(theme: "light" | "dark"): Promise<void> {
11   await AsyncStorage.setItem(KEY, theme);
12 }
```

Error Handling

Async Storage can fail

- Storage is full
- App permissions are restricted
- The device is in a transient error state

Wrap Async Storage calls in `try/catch` to prevent crashes and provide fallback behavior

```
try {  
  await AsyncStorage.setItem(KEY, value);  
} catch (error) {  
  console.error("Failed to save data:", error);  
}
```

Add Error Handling

async-storage-demo/storage/themeStorage.ts

```
2
3 const KEY = "@MyApp_theme";
4
5 export async function loadTheme(): Promise<"light" | "dark"> {
6   try {
7     const saved = await AsyncStorage.getItem(KEY);
8     return saved ? (saved as "light" | "dark") : "light";
9   } catch {
10    return "light";
11  }
12 }
13
14 export async function saveTheme(theme: "light" | "dark"): Promise<void> {
15   try {
16     await AsyncStorage.setItem(KEY, theme);
17   } catch {
18     // Ignore errors
19   }
20 }
```

Load on Startup

In `_layout.tsx`, load saved theme before rendering UI

redux-demo/app/_layout.tsx

```
1 import { Stack } from "expo-router";
2 import { Provider } from "react-redux";
3 import { store } from "../store/store";
4
5 export default function RootLayout() {
6   return (
7     <Provider store={store}>
8       <Stack />
9     </Provider>
10  );
11 }
```

Call async function `loadTheme()`

But async operations cannot run during render

React components must return JSX synchronously

useEffect

React components must return JSX synchronously

- `loadTheme()` returns a **Promise**, we cannot `await` in the render body

`useEffect` runs **after the component mounts**, perfect for reading from Async Storage and updating Redux state

- Ensures the store is updated **after the component exists**, without blocking rendering

Load on Startup

| async-storage-demo/app/_layout.tsx

```
2 import { Stack } from "expo-router";
3 import { Provider } from "react-redux";
4 import { store } from "../store/store";
5 import { setTheme } from "../features/theme/themeSlice";
6 import { loadTheme } from "../storage/themeStorage";
7
8 export default function RootLayout() {
9   useEffect(() => {
10     loadTheme().then((theme) => {
11       store.dispatch(setTheme(theme));
12     });
13   }, []);
14
15   return (
16     <Provider store={store}>
17       <Stack />
18     </Provider>
19   );
20 }
```

- `[]` → run effect only once, on startup

Load on Startup

```
useEffect(() => {  
  loadTheme().then((theme) => {  
    store.dispatch(setTheme(theme));  
  });  
}, []);
```

- `loadTheme()` → returns a **Promise** resolving to `"light"` or `"dark"`
- `.then((theme) => { ... })` → runs after the theme is loaded
- `store.dispatch(setTheme(theme))` → updates Redux store
- `dispatch`: Function used to send actions to the store

Save on Update

When theme changes, persist new value to Async Storage

- Update Redux state & call `saveTheme()` to store theme

redux-demo/features/theme/themeSlice.ts

```
1 import { createSlice } from "@reduxjs/toolkit";
2
3 type ThemeState = { theme: "light" | "dark" };
4
5 const initialState: ThemeState = { theme: "light" };
6
7 const themeSlice = createSlice({
8   name: "theme",
9   initialState,
10  reducers: {
11    toggleTheme: (state) => {
12      state.theme = state.theme === "light" ? "dark" : "light";
13    },
14  },
15 });
16
17 export const { toggleTheme } = themeSlice.actions;
18 export default themeSlice.reducer;
```

Save on Update

| async-storage-demo/feature/theme/themeSlice.ts

```
7
8 const themeSlice = createSlice({
9   name: "theme",
10  initialState,
11  reducers: {
12    setTheme: (state, action: PayloadAction<"light" | "dark">) => {
13      state.theme = action.payload;
14      saveTheme(action.payload);
15    },
16    toggleTheme: (state) => {
17      const newTheme = state.theme === "light" ? "dark" : "light";
18      state.theme = newTheme;
19      saveTheme(newTheme);
20    },
21  },
22 });
23
24 export const { setTheme, toggleTheme } = themeSlice.actions;
25 export default themeSlice.reducer;
```

setTheme

```
setTheme: (state, action: PayloadAction<"light" | "dark">) => {  
  state.theme = action.payload;  
  saveTheme(action.payload);  
},
```

- `PayloadAction<Type>`: TypeScript type from Redux Toolkit
- `import { type PayloadAction } from "@reduxjs/toolkit";`
- Represent an **action object** with a `payload` property of type `Type`

Why Use `PayloadAction`?

- **Type Safety**

```
action: PayloadAction<"light" | "dark">
```

- Only `"light"` or `"dark"` can be passed to this reducer
- TypeScript throws compile-time errors for invalid values
- **Standardizes action structure in Redux Toolkit**
 - Always has a `payload` property
 - Matches what `createSlice` generates automatically

Key Point

```
setTheme: (state, action: PayloadAction<"light" | "dark">) => {
  state.theme = action.payload;
  saveTheme(action.payload);
},
toggleTheme: (state) => {
  const newTheme = state.theme === "light" ? "dark" : "light";
  state.theme = newTheme;
  saveTheme(newTheme);
},
```

Update state first, then persist asynchronously

- `saveTheme()` can fail (Async Storage error), but Redux state is already updated
- Pattern generalizes to any Redux state you want to persist

State Storage Mismatch

Possible Scenario

- `dispatch(setTheme("dark"))` → Redux state is "dark"
- `saveTheme("dark")` fails
- Redux state shows "dark", but Async Storage still has "light"

Such mismatch is ususally acceptable, as it is temporary

- On next app start, load the last successfully saved value
- For small apps, this approach keeps code simple and UI responsive

For Production

Optional Advanced Solutions

- Await Async Storage before updating state

```
try {  
  await saveTheme(newTheme);  
  state.theme = newTheme;  
} catch (error) {  
  // handle error, maybe show a message  
}
```

- Retry or queue failed saves
 - Maintain a retry queue for failed writes
 - Useful for networked storage or critical data
- Show error feedback to users

Async Storage in Redux

- **Load on startup:** In `_layout.tsx`, use `useEffect`
- **Save on update:** Call `saveTheme()` in reducers after state change
- Async and non-blocking — UI updates immediately
- **Key naming:** Use app-specific prefix (`@MyApp_theme`) for clarity and avoiding collisions
- **Error handling:** Wrap Async Storage calls in `try/catch` to prevent app crashes

Assignment 4

- **Load on startup:** In `app/_layout.tsx`, use `useEffect` to load saved activities and dispatch them to Redux
- **Save on update:** In reducers (e.g., `addActivity`), call `saveActivities()` after state updates
- **Key naming:** Use app-specific prefix `@FitnessTracker_activities`
- **Convert for storage:** `storage/activitiesStorage.ts`
 - `loadActivities()`: Use `JSON.parse()`
 - `saveActivities()`: Use `JSON.stringify()`

Where is Data Stored?

- **iOS / macOS**
 - Small values (≤ 1 KB): stored in a shared `manifest.json`
 - Larger values: stored in separate files (named by MD5-hashed `key`)
- **Android / Windows**: Stored in a local SQLite database
- **Web**: Stored in `window.localStorage`

Each app has its own isolated storage area

Data is deleted when the app is uninstalled

Where is Data Stored?

For Expo apps running in iOS simulator

```
~/Library/Developer/CoreSimulator/Devices/<Device-ID>/data/Containers/Data/Application/<App-ID>/Documents/ExponentExperienceData/@anonymous/async-storage-demo-<UUID>/RCTAsyncLocalStorage/
```

- `manifest.json` contains small key-value pairs
- Large values are stored in separate MD5-hashed files

Find simulator's device ID:

```
xcrun simctl list devices
```

To locate `manifest.json`:

```
find ~/Library/Developer/CoreSimulator/Devices/ -name manifest.json
```

Async Storage: API

Async Storage provides a simple asynchronous API for key–value operations

- `setItem(key, value)`: Store a string value
- `getItem(key)`: Retrieve a string value (or null if not found)
- `removeItem(key)`: Delete a stored key and its value

Remove Data

`removeItem(key)`: Remove value for a `key`, invokes (optional) callback once completed

- Commonly used for actions like reset or logout

```
await AsyncStorage.removeItem("@FitnessTracker_activities");
```

Remove Theme

Add a function `clearTheme` to `storage/themeStorage.ts`

```
async-storage-demo/storage/themeStorage.ts
```

```
4
5 export async function loadTheme(): Promise<"light" | "dark"> {
6   try {
7     const saved = await AsyncStorage.getItem(KEY);
8     return saved ? (saved as "light" | "dark") : "light";
9   } catch {
10    return "light";
11  }
12 }
13
14 export async function saveTheme(theme: "light" | "dark"): Promise<void> {
15   try {
16     await AsyncStorage.setItem(KEY, theme);
17   } catch {
18     // Ignore errors
19   }
20 }
21
22 export async function clearTheme(): Promise<void> {}
```

Remove Theme

Remove the saved theme from Async Storage using `removeItem(key)`

| async-storage-demo/storage/themeStorage.ts

```
10     return "light";
11   }
12 }
13
14 export async function saveTheme(theme: "light" | "dark"): Promise<void> {
15   try {
16     await AsyncStorage.setItem(KEY, theme);
17   } catch {
18     // Ignore errors
19   }
20 }
21
22 export async function clearTheme(): Promise<void> {
23   try {
24     await AsyncStorage.removeItem(KEY);
25   } catch {
26     // Ignore errors
27   }
28 }
```

Remove Theme

Add a “Clear Theme” button to [HomeScreen](#)

```
async-storage-demo/app/index.tsx
```

```
7 export default function HomeScreen() {  
8   const theme = useSelector((state: RootState) => state.theme.theme);  
9   const dispatch: AppDispatch = useDispatch();  
10  
11   const handleClearTheme = async () => {  
12     await clearTheme(); // Remove from Async Storage  
13     dispatch(setTheme("light")); // Reset Redux state  
14   };  
15  
16   return (  
17     <View  
18       style={[styles.container, theme === "light" ? styles.light : styles  
19     >  
20       <Text style={styles.text}>Current theme: {theme}</Text>  
21       <Button title="Toggle Theme" onPress={() => dispatch(toggleTheme())  
22       <Button title="Clear Theme" onPress={handleClearTheme} />  
23     </View>  
24   );  
25 }  
26  
27 const styles = StyleSheet.create({
```

Remove w/o API

- Uninstall the app / delete the simulator app
- Delete the storage files manually
- Reset simulator / emulator
- iOS Simulator: Device → Erase All Content and Settings
- Android Emulator: Wipe Data

Async Storage: API

- `setItem(key, value)`: Store a string value
- `getItem(key)`: Retrieve a string value (or null)
- `removeItem(key)`: Delete a stored key and its value
- `mergeItem(key, value)`: Merge an existing `value` stored under `key`, with new `value`
- `getAllKeys()`: Return all keys known to your App
- ...

Doc: Async Storage API

Recap: Async Storage

Asynchronous, persistent key-value storage

- Each item stored under a key (e.g., `@MyApp_theme`)
- Can only store strings: Use `JSON.stringify()` / `JSON.parse()` for objects or arrays
- **API:** `setItem`, `getItem`, `removeItem`, ...
- **Usage Patterns:**
 - **Load on startup:** `useEffect()` in `app/_layout.tsx` to populate Redux store
 - **Save on update:** Call async save functions in reducers (`setTheme`, `addActivity`, etc.)

Async Storage: Limitation

Not encrypted: all data is stored in plain text

- Good for: App state, preferences, caching
- Not for: Sensitive data such as passwords, tokens

Sensitive data should always be stored securely

Expo SecureStore

Encrypted storage for iOS and Android

- Encrypts and securely stores key-value pairs locally on the device
- Each Expo project has its own isolated storage
- No cross-project access

Documentation: Expo SecureStore

Use Together

- **Async Storage:** Non-sensitive data (app state, preferences)
- **SecureStore:** Sensitive data (passwords, tokens)

```
import AsyncStorage from "@react-native-async-storage/async-storage";
import * as SecureStore from "expo-secure-store";

// Store general app data
await AsyncStorage.setItem("@MyApp_theme", "dark");

// Store sensitive token
await SecureStore.setItemAsync("userToken", token);
```

Keep UI data fast with Async Storage and protect secrets with SecureStore

Lecture Summary

Redux Toolkit: Scalable global state for larger apps

- **Slices:** Organize state + reducers + actions
- **Provider + hooks:** Any component can read or update state using `useSelector`, `useDispatch`
- **Immer:** Enables “mutating” syntax for immutable updates

Lecture Summary

Data Persistence

- **Async Storage:** Asynchronous, persistent key-value storage for React Native
- **API:** `setItem`, `getItem`, `removeItem`, ...
- **Error handling:** `try/catch`
- **Integration with Redux:** Load on app start (`app/_layout.tsx`); save after updates (reducers)
- **Expo SecureStore:** Encrypted storage for iOS and Android

Speaker notes